

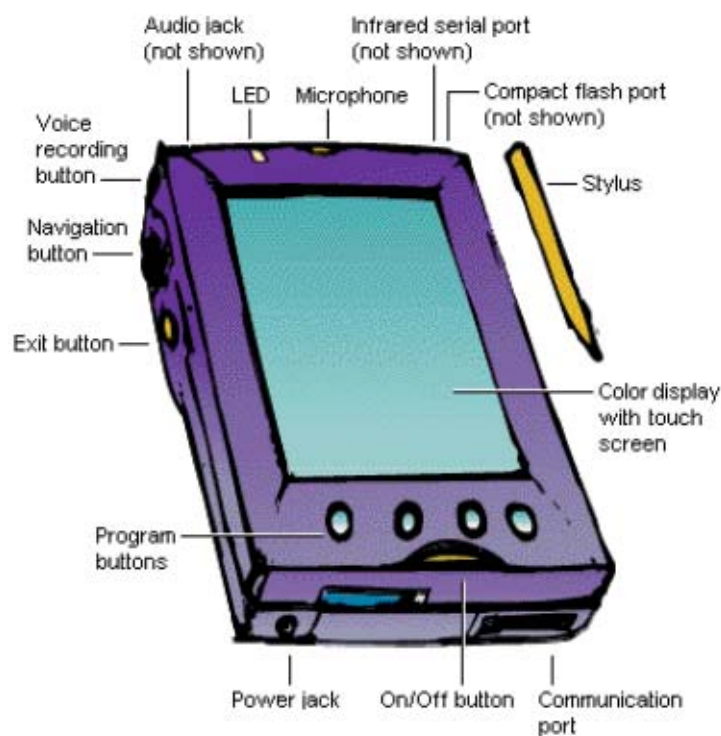
## บทที่ 2

### สถาปัตยกรรมของ Pocket PC

Pocket PC ถือได้ว่าเป็น PDAs (Personal Digital Assistants) หรือคอมพิวเตอร์แบบพกพาอีกแบบหนึ่ง ซึ่งบริษัทไมโครซอฟท์ผู้เป็นเจ้าของเครื่องหมายการค้าได้นิยามไว้ว่า Pocket PC คือ “อุปกรณ์มือถือที่ผู้ใช้สามารถเก็บและเรียกข้อมูลทั้งอีเมล บันทึกการติดต่อ บันทึกการนัดหมาย เล่นไฟล์บันทึก เล่นเกม สื่อสารกันด้วยข้อความด้วย MSN Messenger ท่องเว็บไซต์ และอื่นๆ”

ในอีกกรณีหนึ่ง เราสามารถกล่าวได้ว่าอุปกรณ์ใดๆ ซึ่งถูกเรียกว่า “Pocket PC” จะต้อง

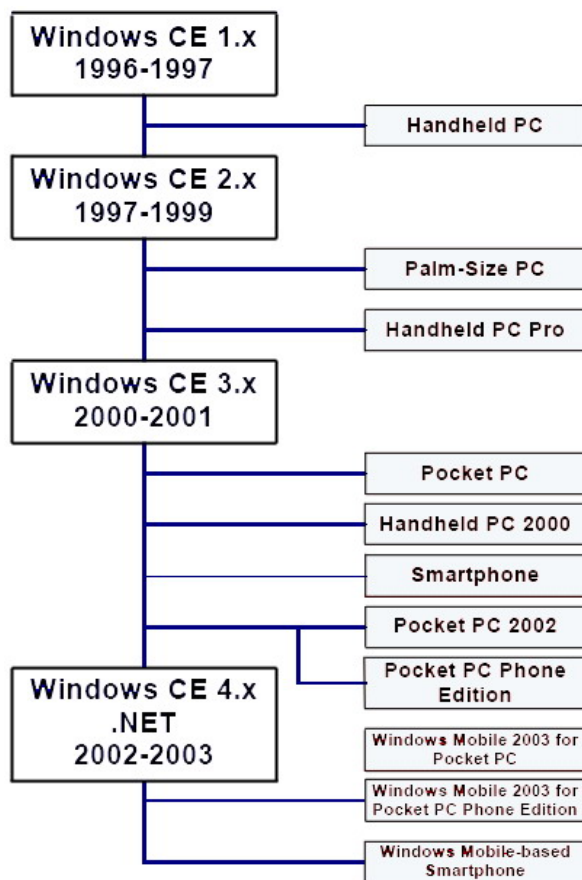
- ทำงานโดยใช้ระบบปฏิบัติการ Windows CE ของไมโครซอฟท์
- รวมชุดของแอปพลิเคชันไว้ใน ROM
- มีจอสัมผัส (Touch Screen)
- มีปุ่มบังคับทิศทางหรือปุ่มแบบสัมผัส
- มีชุดของปุ่มเพื่อสั่งการแอปพลิเคชัน
- มีพื้นฐานของหน่วยประมวลผลกลางแบบ ARM 4 CPU, Intel XScale CPU, MIPS CPU หรือ SH3 CPU



รูปที่ 2.1 ส่วนประกอบของ Pocket PC

## 2.1 ระบบปฏิบัติการใน Pocket PC

ไมโครซอฟท์นั้นใช้ระบบปฏิบัติการ Windows CE เป็นพื้นฐานใน Pocket PC โดยวิวัฒนาการของ Windows CE เป็นดังรูปที่ 2.2

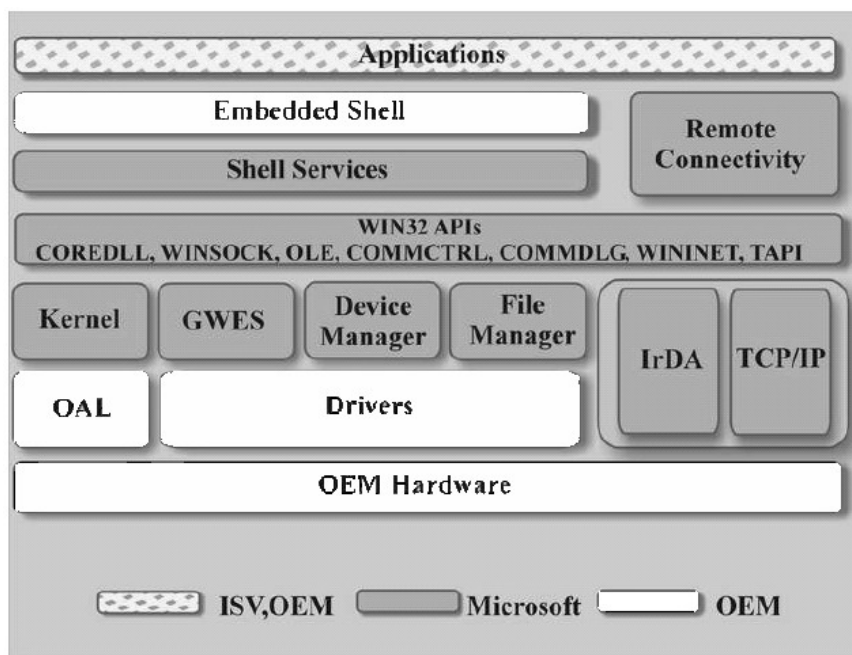


รูปที่ 2.2 วิวัฒนาการของ Windows CE

โดยที่ระบบปฏิบัติการใน Pocket PC รุ่นล่าสุดและเป็นระบบปฏิบัติการที่ใช้ในการทำโครงการนี้คือ Windows Mobile 2003 Second Edition ซึ่งใช้แกนหลักของระบบเป็น Windows CE 4.21 และมีลักษณะการทำงานเป็น .NET Compact Framework ดังจะอธิบายในหัวข้อย่อต่อไป

### 2.1.1 สถาปัตยกรรมของ Windows CE

สถาปัตยกรรมของระบบปฏิบัติการ Windows CE เป็นดังรูปที่ 2.3 ส่วนที่เป็นสีเทานั้นคือส่วนที่ออกแบบโดยไมโครซอฟท์ โดยที่ทางบริษัทผู้ผลิตหรือ OEM (Original Equipment Manufacturer) ไม่ได้ได้รับอนุญาตให้เปลี่ยนแปลงโครงสร้างส่วนนั้นได้ ส่วนที่เป็นสีขาวเป็นส่วนในความรับผิดชอบของ OEM และเฉพาะผู้ผลิตซอฟต์แวร์อิสระหรือ ISV (Independent Software Vendor) หรือทาง OEM เท่านั้นที่จะสามารถสร้างส่วนของแอปพลิเคชันได้



รูปที่ 2.3 สถาปัตยกรรมของ Windows CE

ในชั้นของแอปพลิเคชันแสดงให้เห็นว่าแอปพลิเคชันฝังจะถูกเขียนโดยผู้พัฒนา ในส่วนของ Embedded Shell ในอุปกรณ์ที่ทำงานด้วย Windows CE ได้ จะคล้ายคลึงกับเดสก์ท็อปในคอมพิวเตอร์แบบตั้งโต๊ะ จากนั้นส่วนของ Shell Services จะประกอบไปด้วย การสร้างเมนู การเพิ่มไอคอนบนทาสก์บาร์ และการแสดงรายชื่อเอกสารล่าสุดในเมนูเริ่มต้น โดยที่ Shell Services นั้นเป็นส่วนตอบโต้ระหว่างแอปพลิเคชันฝังและส่วนย่อยของ WIN32 Application Programming Interfaces (API) โดยที่ฟังก์ชัน API เป็นส่วนของ รูทีน, โปรโตคอล และเครื่องมือที่จำเป็นในการสร้างซอฟต์แวร์ใดๆ

ส่วนของ Remote Connectivity นั้นจะเป็นส่วนที่จัดการแอปพลิเคชันของระบบเพื่อติดต่อกับ PC โดยช่องทางผ่าน USB รวมทั้งเครือข่ายเคลื่อนที่ และการเข้าถึงอินเทอร์เน็ตด้วย

ส่วนของ GWES (Graphics, Windowing and Event Subsystem) เป็นส่วนที่จัดการฟังก์ชันหน้าจอตอบโต้กับผู้ใช้และความสามารถทางด้านกราฟิกของ Windows CE ซึ่งเป็นเซิร์ฟเวอร์ระบบอันซับซ้อนที่จัดการกับอินพุทของผู้ใช้, เอาท์พุทของแอปพลิเคชัน, การส่งอีเวนต์เมสเสจ และ Windowing

ในส่วน File Manager จะทำการสนับสนุนระบบไฟล์, ฐานข้อมูลที่ใช้สร้าง และรีจิสทรีของระบบ

ส่วนของการติดต่อสื่อสาร 2 แบบในสถาปัตยกรรมนั้นมี 2 โปรโตคอลทางเครือข่ายคือ TCP/IP และ IrDA (Infrared Data Association) โดยทั้ง 2 แบบจะเข้าถึงแอปพลิเคชันผ่านทาง WinSock และ WNet APIs

ส่วน Kernel ถือได้ว่าเป็นส่วนประกอบหลักที่สนับสนุนแกนฟังก์ชันของ Windows CE โดยจะทำการติดต่อกับการจัดการการประมวลผลและเชรค, การจัดตาราง, การจัดการหน่วยความจำ และการ Exception Handling ซึ่งส่วนประกอบนี้จะต้องใช้ได้ในทุกๆ แพลตฟอร์มและการปรับแต่ง โดยจะเขียนโดยไมโครซอฟท์และให้จัดการในทุกๆ CPU ที่สนับสนุน

ส่วน Device Manager จะใช้ในการควบคุมอุปกรณ์ฮาร์ดแวร์ที่ต่อเข้ามา ผู้ใช้สามารถเพิ่มอุปกรณ์พิเศษที่สนับสนุนด้วยไดรฟ์เวอร์อุปกรณ์ที่ใช้ได้ โดยจะถูกเรียกใช้และยกเลิกโดย Device Manager

ในส่วนบนของ OEM Hardware จะเกาะกับ OAL (OEM Adaptation Layer) โดยจะจัดการในส่วนตอบโต้ระหว่าง Kernel และฮาร์ดแวร์ ซึ่งส่วนตอบโต้นี้นั้นประกอบด้วยหมายเลขของรูทีน, ส่วนการสนับสนุน, ISR (Interrupt Service Routines), RCT (Real-Time Clock), ตัวจับเวลาภายใน, การดีบั๊ก, การเปิดปิดอินเทอร์รัพท์, ส่วนข้อมูล และอื่นๆ อีกมาก

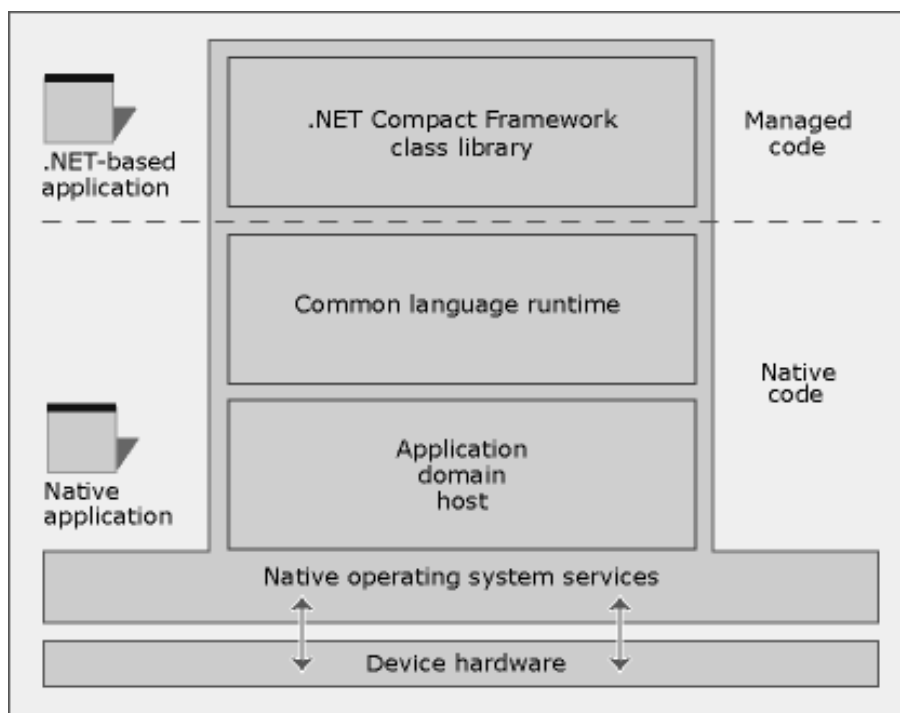
### 2.1.2 .NET Compact Framework

.NET Compact Framework เป็นสภาวะแวดล้อมที่ได้ถูกออกแบบมาให้มีสมรรถนะที่เหมาะสมที่สุดภายใต้อุปกรณ์ที่มีทรัพยากรเพียงจำกัดอย่าง Pocket PC ดังจะเห็นได้จากส่วนต่างๆ ดังนี้

#### 1) สถาปัตยกรรมของ .NET Compact Framework

.NET Compact Framework นั้นเป็นส่วนที่ทำให้แอปพลิเคชันเรียกใช้ระบบปฏิบัติการของอุปกรณ์นั้นๆ ได้อย่างมีประสิทธิภาพ โดยจะสนับสนุนและเรียกใช้บริการต่างๆ ของระบบปฏิบัติการพื้นฐาน (Native Operating System Services) ด้วยความสามารถในการเลือกเฟ้นและจะอ้างถึง API พื้นฐานอย่างถูกต้อง

แอปพลิเคชันในการจัดการและแอปพลิเคชันพื้นฐาน (Native Application) นั้นจะทำงานในเวลาเดียวกันได้ โดยที่โฮสต์ของแอปพลิเคชันโดเมน (Application Domain Host) ซึ่งเป็นแอปพลิเคชันพื้นฐานในตัวเองจะเริ่มเรียกใช้ Common Language Runtime เพื่อทำการรันโค้ดเพื่อการจัดการ (Managed Code) ต่อไป รูปที่ 2.4 แสดงถึงสถาปัตยกรรมโดยรวมของ .NET Compact Framework



รูปที่ 2.4 สถาปัตยกรรมโดยรวมของ .NET Compact Framework

## 2) แอปพลิเคชันโดเมน

ทุกๆ แอปพลิเคชันของ .NET Compact Framework ทำงานภายในส่วนของโครงสร้างการทำงานที่เรียกว่าแอปพลิเคชัน โดเมน ซึ่งจะคล้ายกับการประมวลผลของระบบปฏิบัติการ โดยที่ .NET Compact Framework จะต้องแน่ใจว่าทรัพยากรในการจัดการทั้งหมดที่ได้ถูกใช้โดยแอปพลิเคชันที่มีการทำงานนั้นว่างลงหรือมีการส่งค่ากลับไปยังระบบปฏิบัติการแล้ว เมื่อการทำงานของแอปพลิเคชันนั้นสิ้นสุดลง

การมีแอปพลิเคชันโดเมนนั้นมีประโยชน์มากมายในการประมวลผล ยกตัวอย่างเช่น มีการแยกความผิดพลาด, ช่วยเพิ่มความสมบูรณ์, มีความปลอดภัย โดยที่ไม่จำเป็นต้องร้องขอจากระบบปฏิบัติการที่อยู่ในตำแหน่งที่ต่ำกว่า โดยที่โฮสต์ของแอปพลิเคชันโดเมนจะเรียกใช้ Common Language Runtime และโค้ดของระบบปฏิบัติการพื้นฐานเอง ซึ่ง Common Language Runtime นั้นจะสามารถติดต่อกับโฮสต์ของแอปพลิเคชันโดเมนได้ทั้งแบบสแตติกและแบบไดนามิก

อย่างไรก็ตาม .NET Compact Framework นั้นไม่ได้ทำการจำกัดพฤติกรรม (Behavior) ของโฮสต์ของแอปพลิเคชันโดเมนแต่อย่างใด โฮสต์ของแอปพลิเคชันโดเมนจะสามารถทำการเพิ่มเข้าไปยังเซลล์สื่อสารที่มีอยู่ที่ใช้ในการเริ่มต้นหรือหยุดยังโปรแกรมได้ บนระบบแอปพลิเคชันแบบไดนามิก เช่น Windows โฮสต์ของแอปพลิเคชันโดเมนจะสามารถเพิ่มเข้าไปยังตัวโหลดของแอปพลิเคชันได้ ดังนั้นแอปพลิเคชันของ .NET Compact Framework จึงสามารถเริ่มต้นและหยุดยังได้โดยใช้กลไกเดียวกับในแอปพลิเคชันพื้นฐาน

### 3) แอปพลิเคชันโดเมนแบบหลายตัว

.NET Compact Framework นั้นสนับสนุนแอปพลิเคชันโดเมนแบบหลายตัว โดยสามารถกำหนดโค้ดแอสเซมบลีลงในโครงสร้างของคลาสได้ จากนั้นจะสามารถใช้ คลาส System.AppDomain.ExecuteAssembly ในการสร้างแอปพลิเคชันโดเมนตัวใหม่ โดยที่แอปพลิเคชันโดเมนตัวใหม่นั้นจะโหลดสำเนาของ Common Language Runtime DLLs, โครงสร้างข้อมูล และกลุ่มของหน่วยความจำด้วยตัวของมันเอง แอปพลิเคชันโดเมนแบบหลายตัวนั้นจะสามารถอยู่ในการประมวลผลของระบบปฏิบัติการหนึ่งตัวได้

การเริ่มใช้งานแอปพลิเคชันโดเมนตัวใหม่จะเป็นการเรียกใช้แบบซิงโครนัสในสถานะแวดล้อมของแอปพลิเคชันโดเมนดั้งเดิม ดังนั้นจึงจำเป็นต้องเริ่มเซดใหม่ถ้าแอปพลิเคชันโดเมนดั้งเดิมต้องการการทำงานร่วมกันแอปพลิเคชันโดเมนแบบหลายตัว และอย่างไรก็ตามไม่เหมือนใน .NET Framework แบบตัวเต็ม ปัจจุบัน .NET Compact Framework นั้นยังไม่สนับสนุนการโหลดโค้ดแอสเซมบลีลงในพื้นที่โค้ดกลางของโดเมนเพื่อใช้แอปพลิเคชันโดเมนแบบหลายตัว

การทำงานของ .NET Compact Framework จะสิ้นสุดลงเมื่อกลุ่มขยะ (Garbage Collection) เกิดมีการทำงานขึ้น กลุ่มขี้นั้นอาจจะเกิดขึ้นในแอปพลิเคชันโดเมนเดี่ยวๆ หรือในทุกๆ แอปพลิเคชันโดเมนก็ได้ ทั้งนี้เพื่อเป็นการป้องกันแอปพลิเคชันโดเมนหนึ่งจากการใช้หน่วยความจำมากเกินไปจนไม่เหลือให้การทำงานอื่น

### 4) ส่วนจัดการหน่วยความจำ

จุดแข็งที่สำคัญอย่างมากของ .NET Compact Framework นั่นก็คือการจัดการทรัพยากรได้อย่างมีประสิทธิภาพ โดยเฉพาะอย่างยิ่งกับ RAM ที่มีการเปลี่ยนแปลงได้ง่าย โดยที่อุปกรณ์นั้นไม่จำเป็นต้องมีหน่วยจัดการหน่วยความจำของฮาร์ดแวร์หรือหน่วยความจำเสมือนในระบบปฏิบัติการแต่อย่างใด

.NET Compact Framework ได้ถูกออกแบบให้ระบบมีการปฏิบัติการได้อย่างมีประสิทธิภาพที่สุดด้วยลักษณะเฉพาะดังต่อไปนี้

- ใช้พลังงานของแบตเตอรี่อย่างคุ้มค่า
- มีความสามารถในการ Flash หน่วยความจำหรือระบบไฟล์ของ RAM 5-10 ครั้งโดยประมาณ เพื่อการเก็บ .NET Compact Framework และแอปพลิเคชันต่างๆ ลงใน RAM แบบไดนามิก (DRAM)
- มีที่ว่างในการทำงานจากเล็กที่สุด 128 KB ถึง 1 MB ใน DRAM
- ใช้ฮาร์ดไดรฟ์ช่วยในการทำงาน

.NET Compact Framework มีการจัดการการใช้หน่วยความจำของระบบอย่างระมัดระวังที่สุด โดยจะไม่มีการใช้ RAM หากไม่มีการเรียกแอปพลิเคชันให้ทำงาน ยิ่งไปกว่านั้น .NET Compact Framework จะทำการล้างข้อมูลใน RAM ให้ว่างทันทีที่ออกจากโปรแกรม โดยที่ระบบปฏิบัติการพื้นฐานนั้นไม่จำเป็นต้องมีความสามารถในการจัดการกับ RAM ในตัวของมันเอง ยกเว้นเมื่อมีการเข้าถึงหน่วยความจำที่ไม่ใช่ของตัวเอง

เมื่อหน่วยความจำเกิดการขาดแคลน .NET Compact Framework จะจัดการปลดปล่อยโครงสร้างข้อมูลภายในที่ไม่จำเป็นในทันทีโดยโค้ดที่กำลังทำงานอยู่ ดังนั้นโปรแกรมก็ยังสามารถทำงานต่อไปได้แม้ว่าจะเกิดสถานการณ์ที่หน่วยความจำเหลือน้อยก็ตาม และถ้าแอปพลิเคชันใดเกิดต้องการใช้หน่วยความจำมากเกินไป .NET Compact Framework ก็จะปิดแอปพลิเคชันนั้นอย่างเรียบร้อยและคืนทรัพยากรสู่ตำแหน่งเดิมทั้งหมด ที่สำคัญ .NET Compact Framework นั้นจะไม่มีทางหยุดทำงานแม้หน่วยความจำเหลือไม่มากพอ

โฮสต์ของแอปพลิเคชัน โดเมนจะเป็นผู้เริ่มการทำงานของแอปพลิเคชัน .NET Compact Framework และ Common Language Runtime โดยแอปพลิเคชันของ .NET Compact Framework จะใช้โค้ดสเปซและสเปซข้อมูลทั้งแบบสแตติกและไดนามิกในรูปแบบเดียวกับแอปพลิเคชันพื้นฐาน ในเวลาที่แอปพลิเคชันของ .NET Compact Framework ไม่มีการทำงาน จะไม่มีการใช้ RAM นอกจากโฮสต์ของแอปพลิเคชัน โดเมน และจะมีข้อมูลแบบสแตติกเพียงจำนวนเล็กน้อยเท่านั้นที่จะถูกใช้โดย Common Language Runtime โดยระบบปฏิบัติการ Windows CE .NET จะสร้างโฮสต์ของแอปพลิเคชัน โดเมนขึ้นเมื่อแอปพลิเคชันของ .NET Compact Framework จะเริ่มทำงาน

แอปพลิเคชันของ .NET Compact Framework นั้นจะเป็นแฟ้มเอกที่รวมไฟล์สกุล .EXE และ .DLL ไว้ ซึ่งจะสามารถเก็บลงในระบบไฟล์ทั้งแบบอ่านอย่างเดียวหรือแบบทั้งอ่านและเขียนใน Flash (หรือใน RAM ในแบบอ่านอย่างเดียว) ตัวโหลดคลาสของ Common Language Runtime จะสามารถอ่านไฟล์เหล่านั้นในบล็อกที่อยู่ได้โดยตรงโดยไม่ต้องทำสำเนาในหน่วยความจำและไม่ต้องการหน่วยจัดการหน่วยความจำในการสร้างมุมมองแผนที่หน่วยความจำ (Memory-Mapped View) ของไฟล์แต่อย่างใด

### 5) ส่วนความปลอดภัย

.NET Compact Framework ได้มีส่วนย่อยของความปลอดภัยในการเข้าถึงโค้ด และมีแผนที่จะขยายคลาสเพื่อปรับตัวให้เข้ากับความเป็นจริงของการรักษาความปลอดภัยในอุปกรณ์เคลื่อนที่ในอนาคต แผนการทางด้านรักษาความปลอดภัยในปัจจุบันและอนาคตของ .NET Compact Framework มีดังต่อไปนี้

- มีนโยบายการรักษาความปลอดภัยในการยับยั้งการจัดการกับโค้ดในแต่ละอุปกรณ์ อย่างไรก็ตาม หากแพลตฟอร์มนั้นยังคงสนับสนุนการทำโค้ดพื้นฐาน แพลตฟอร์มในการรักษาความปลอดภัยก็ยังไม่สามารถทำได้เต็มที่
- โสตร์ของแอปพลิเคชันโคเมนจะมีการแสดงการพิสูจน์หลักฐานที่เชื่อถือได้เกี่ยวกับสถานที่ที่โค้ดได้ถูกโหลดลงไปแล้ว อาจจะเป็นไปได้ที่จะมีการเพิ่มเงื่อนไขสมาชิกของกลุ่มโค้ดที่เพิ่มเข้ามา อย่างที่มีใน .NET Framework แบบตัวเต็ม
- มีการร้องขอความยินยอมที่ยืนยันได้ที่ไม่เป็นเวอร์ชันแรก ซึ่งไปประกอบด้วยแอททริบิวต์ RequestMinimum, RequestOptional และ RequestRefuse ที่ใช้ในการกรอกรนโยบายการรักษาความปลอดภัยเมื่อเวลาที่มีการโหลด
- มีการพิสูจน์หลักฐานของ Custom Assembly และนโยบายความปลอดภัยที่ถูกแทนที่ได้
- มี Custom Permission Type เพื่อวางแผนการรักษาความปลอดภัยใน Mobile Extension ของ .NET Compact Framework
- Custom Permission Type จะเริ่มจากผู้ผลิตอุปกรณ์นั้นๆที่จะมีฟังก์ชันข้อกำหนดทางด้านการรักษาความปลอดภัยของแพลตฟอร์ม และ Custom Permission นั้นจะหา Namespace เพื่อเพิ่มลงใน .NET Compact Framework

## 2.2 หน่วยความจำใน Pocket PC

หน่วยเก็บความจำภายในที่ติดตั้งมากับ Pocket PC นั้นจะประกอบด้วยหน่วยความจำ 2 ส่วนคือ ROM และ RAM ซึ่งจะมี ROM อย่างน้อย 24 เมกกะไบต์ และ RAM อย่างน้อย 16 เมกกะไบต์ ซึ่งส่วนประกอบของระบบปฏิบัติการนั้นจะถูกบีบอัดลงใน ROM และเมื่อผู้ใช้เรียกใช้ส่วนประกอบส่วนใด ส่วนประกอบนั้นก็จะคลายออกแล้วส่งต่อไปยัง RAM

**1) ROM (Read-Only Memory)** เป็นหน่วยความจำที่ใช้อ่านได้อย่างเดียว เป็นหน่วยความจำที่ทำหน้าที่เก็บแอปพลิเคชันและข้อมูลแบบถาวร และไม่ต้องใช้กระแสไฟฟ้าไปเลี้ยงระบบ ซึ่งแอปพลิเคชันและข้อมูลจะถูกเขียนหรือเบิร์นเข้าไปในชิปโดยบริษัทผู้ผลิต โดยผู้ใช้จะไม่สามารถเปลี่ยนแปลงข้อมูลที่อยู่ในชิปได้

ส่วนของ ROM นั้นถือได้ว่าเป็นสิ่งจำเป็นที่สุดสำหรับคอมพิวเตอร์แบบพกพาทุกแบบ เพราะว่าคอมพิวเตอร์แบบพกพาส่วนใหญ่นั้นจำเป็นจะต้องบรรจุโปรแกรมของระบบปฏิบัติการและแอปพลิเคชันบางอย่างไว้ใน ROM และจะถูกเรียกใช้ทันทีที่เปิดเครื่อง

ส่วนใหญ่ ROM ใน Pocket PC ในปัจจุบันจะเป็นแบบ Flash ROM ซึ่งจะสามารถทำการอัปเดต ROM เพื่อเพิ่มความสามารถของระบบปฏิบัติการได้ ซึ่งต่างจาก ROM แบบธรรมดาในสมัยก่อนซึ่งการอัปเดต ROM นั้นทำได้ยาก

ในส่วนของ Flash ROM ยังสามารถแยกออกเป็น NAND Flash ROM และ NOR Flash ROM ซึ่งคุณลักษณะของ NAND Flash ROM ก็คือ มีราคาถูก เก็บข้อมูลได้มาก และมีอายุการใช้งานได้นานกว่า NOR Flash ROM ซึ่งเป็นเทคโนโลยีแบบเก่าถึง 10 เท่า ในส่วนของข้อดีของ NAND Flash ROM ก็คือ จะมีความเร็วในการจัดการข้อมูลช้ากว่า เพราะเป็นโครงสร้างการจัดการแบบ XIP (Execute In Place) กล่าวคือ กระบวนการทำงานของเครื่องที่ใช้ NAND Flash ROM จำเป็นจะต้องอ่านข้อมูลจาก NAND Flash ROM ก่อน แล้วค่อยเรียกขึ้นไปทำงาน คล้ายๆ กับการทำงานของฮาร์ดดิสก์ใน PC ดังนั้นส่วนของหน่วยความจำหลักที่ใช้ NAND Flash ROM จำเป็นจะต้องมีขนาดค่อนข้างใหญ่ เพื่อใช้สำรองระบบการจัดการข้อมูลแบบนี้ จึงทำให้พื้นที่ของส่วนความจำเพื่อใช้เก็บโปรแกรมและข้อมูลอื่นๆ จะเหลือน้อยลง นอกจากนี้ใน Pocket PC บางรุ่นยังยอมให้ผู้ใช้สามารถเก็บข้อมูลไว้ใน Flash ROM ได้อีกด้วย ทำให้เวลาเมื่อมีการทำ Hard Reset ข้อมูลที่เราเก็บไว้ใน Flash ROM ก็จะไม่สูญหายไป

2) **RAM (Random Access Memory)** เป็นหน่วยความจำที่จำเป็นของคอมพิวเตอร์ทุกๆ ไป รวมทั้ง Pocket PC ด้วย ส่วนของ RAM นี้ จะใช้สำหรับเก็บโปรแกรมและไฟล์ข้อมูล RAM มีความแตกต่างจาก ROM ในเรื่องของการเก็บรักษาข้อมูล RAM จำเป็นจะต้องมีกระแสไฟมาเลี้ยงเพื่อที่จะรักษาโปรแกรมหรือไฟล์ข้อมูลที่เก็บไว้ใน RAM ดังนั้นผู้ใช้ Pocket PC จำเป็นจะต้องคอยดูแลส่วนของแบตเตอรี่ไม่ให้กระแสไฟฟ้าหมด มิเช่นนั้น โปรแกรมและข้อมูลต่างๆ ที่เก็บไว้ใน RAM ก็ จะหายไป